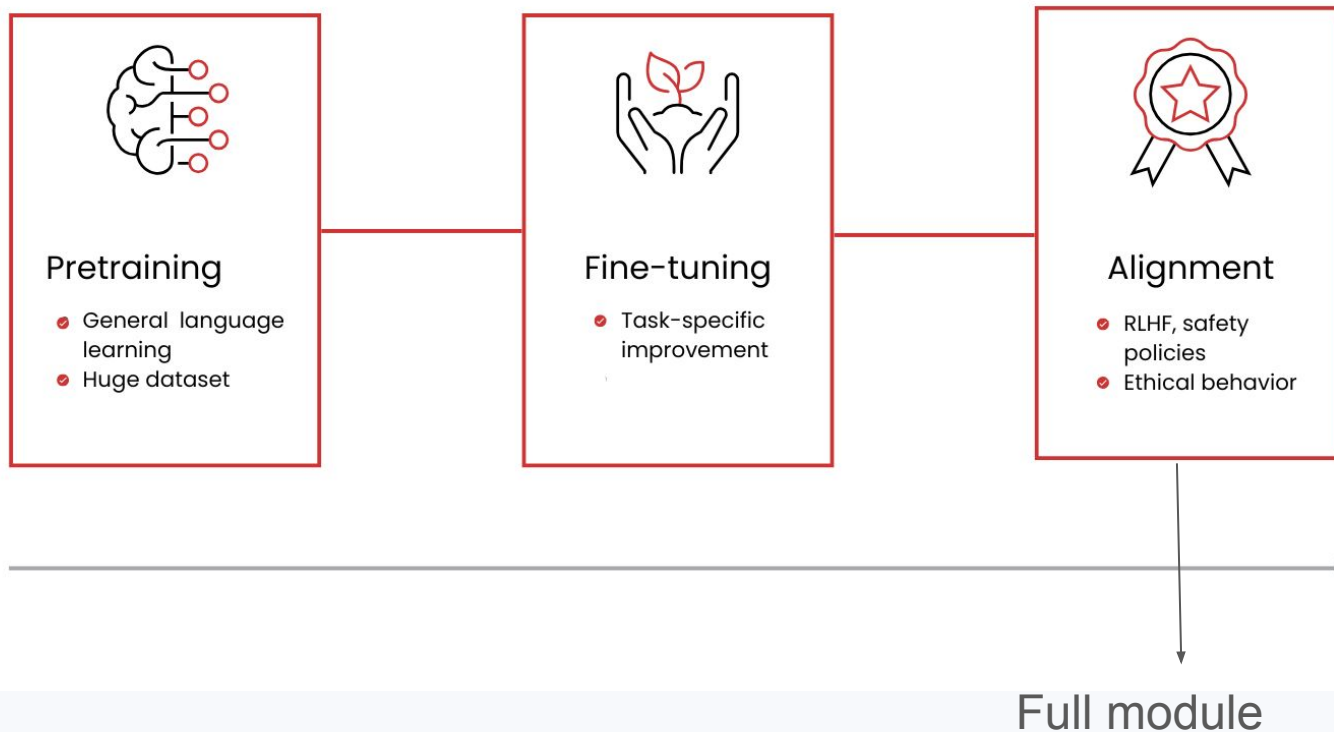


LLM Training

Agenda

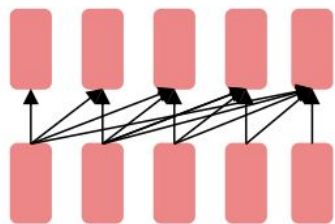
- LLM training phases
- Pre-training
- Fine tuning

LLM TRAINING PIPELINE



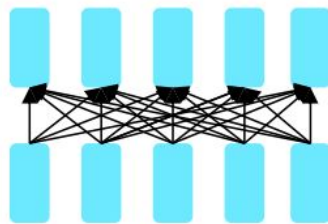
Pre-Training

Reminder: There are three architectures for LLMs



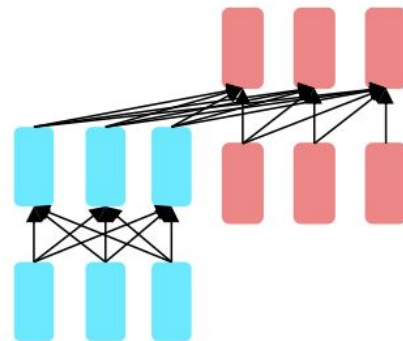
Decoders

GPT, Claude,
Llama
Mixtral



Encoders

BERT family,
HuBERT



Encoder-decoders

Flan-T5, Whisper

Encoders

Popular: Masked Language Models (MLMs)

BERT family:

- Trained by predicting words from surrounding words on both sides
- Are usually finetuned (trained on supervised data) for classification tasks.

Encoders-Decoders

Trained to map from one sequence to another

Very popular for:

- machine translation (map from one language to another)
- speech recognition (map from acoustics to words)

Decoders

Predict words left to right

Also called:

- Causal LLMs
- Autoregressive LLMs
- Left-to-right LLMs

Use All the Knowledge in the Text

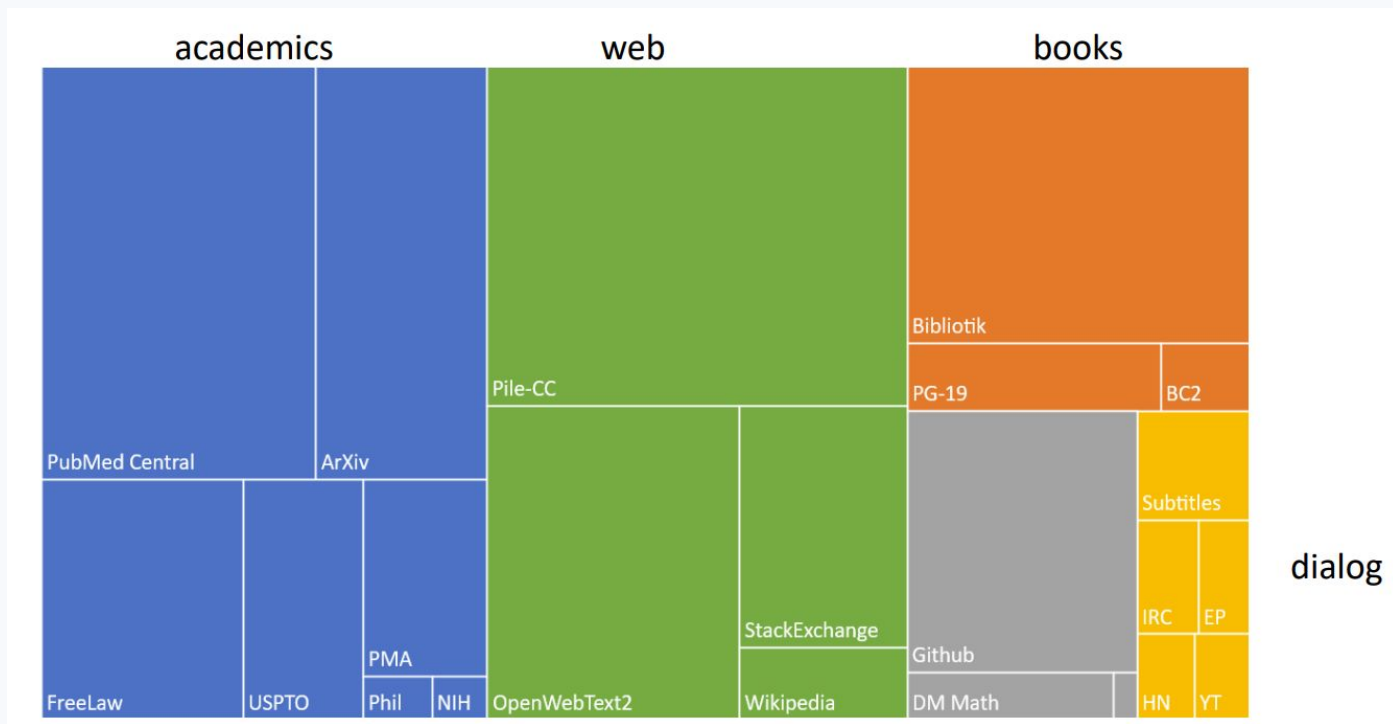
The big idea that underlies all the amazing performance of language models:

- First pretrain a transformer model on **enormous** amounts of text
- Then apply it to new tasks.

Text contains enormous amounts of knowledge

Pretraining on lots of text with all that knowledge is what gives language models their ability to do so much

Data for Training



But there are problems with scraping from the web

- **Copyright:**

- much of the text in datasets is copyrighted
- remains an open legal question

- **Data consent:**

- Website owners can indicate they don't want their site crawled

- **Privacy:**

- Websites can contain private IP addresses and phone numbers

Data for Training - Filtering for Quality and Safety

Quality

- Duplication at many levels (URLs, documents, even lines)

Safety

- Toxicity handling is important
- Need to remove adult content

Bias

- Need to handle bias

Loss Function

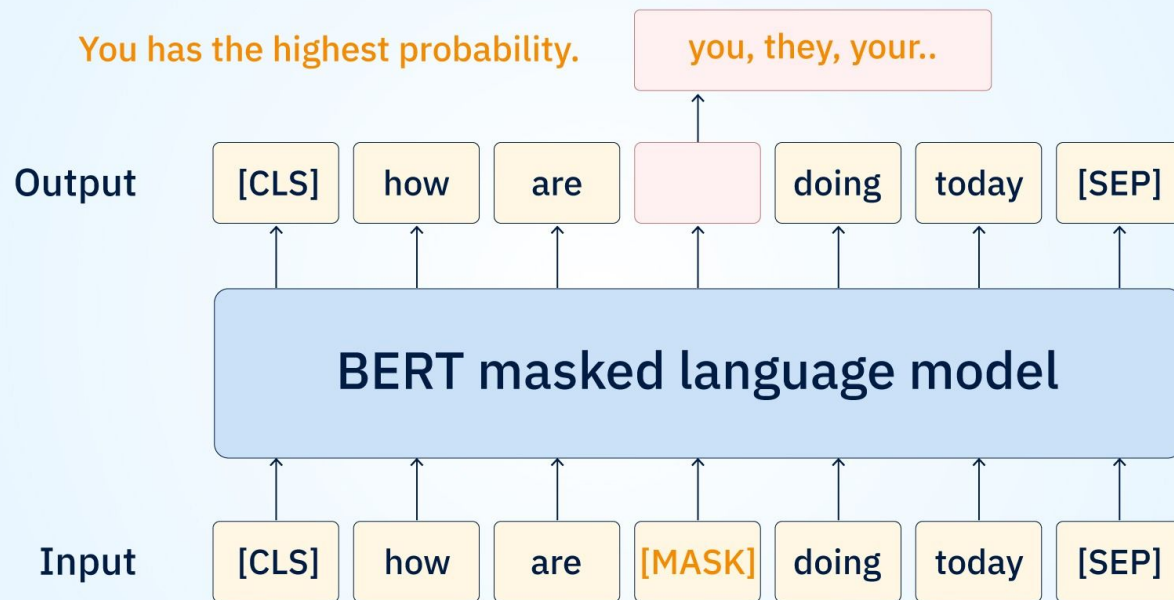
- Same loss function: cross-entropy loss
 - We want the model to assign a high probability to true word w
 - want loss to be high if the model assigns too low a probability to w

Self-Supervised Learning

"Self-supervised" because it just uses the next \ missing word as the label!

Masked LM - BERT

BERT – masked LM (MLM)



BERT – Next Sentence Prediction (NSP)

Sentence 1

Sentence 2

Next Sentence?

I am going outside.

I will be back after 6.

YES

I am going outside.

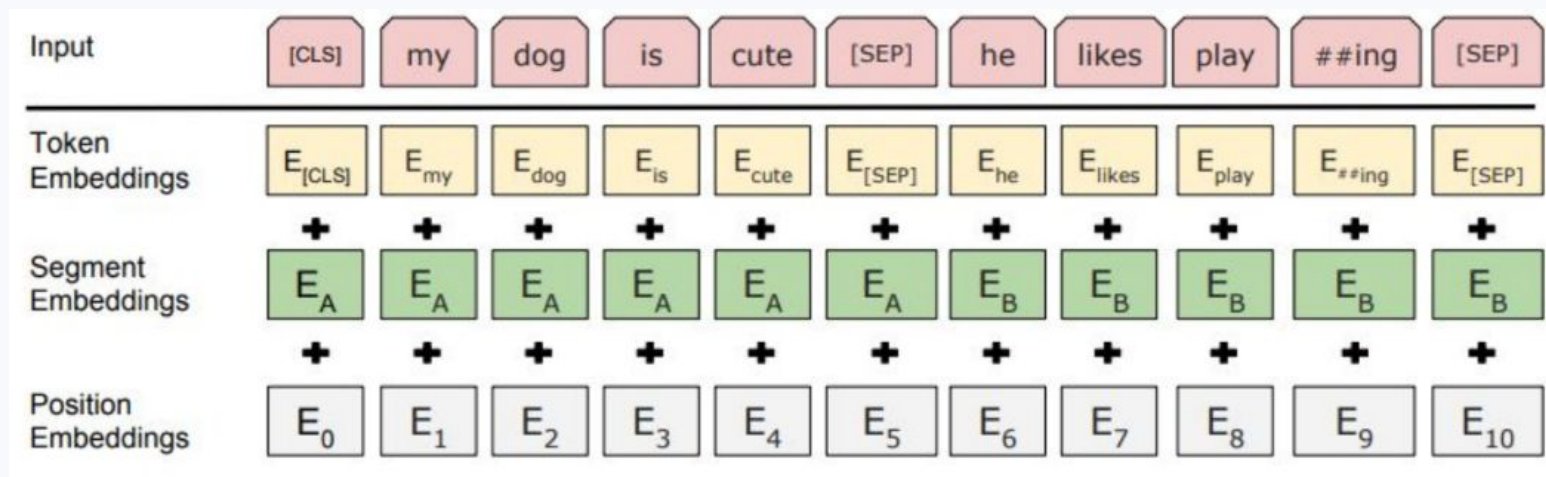
You know nothing John snow.

NO

BERT – Combined Loss

MLM + NSP

BERT - Special Tokens



From
BERT

Causal LM - GPT

Teacher forcing

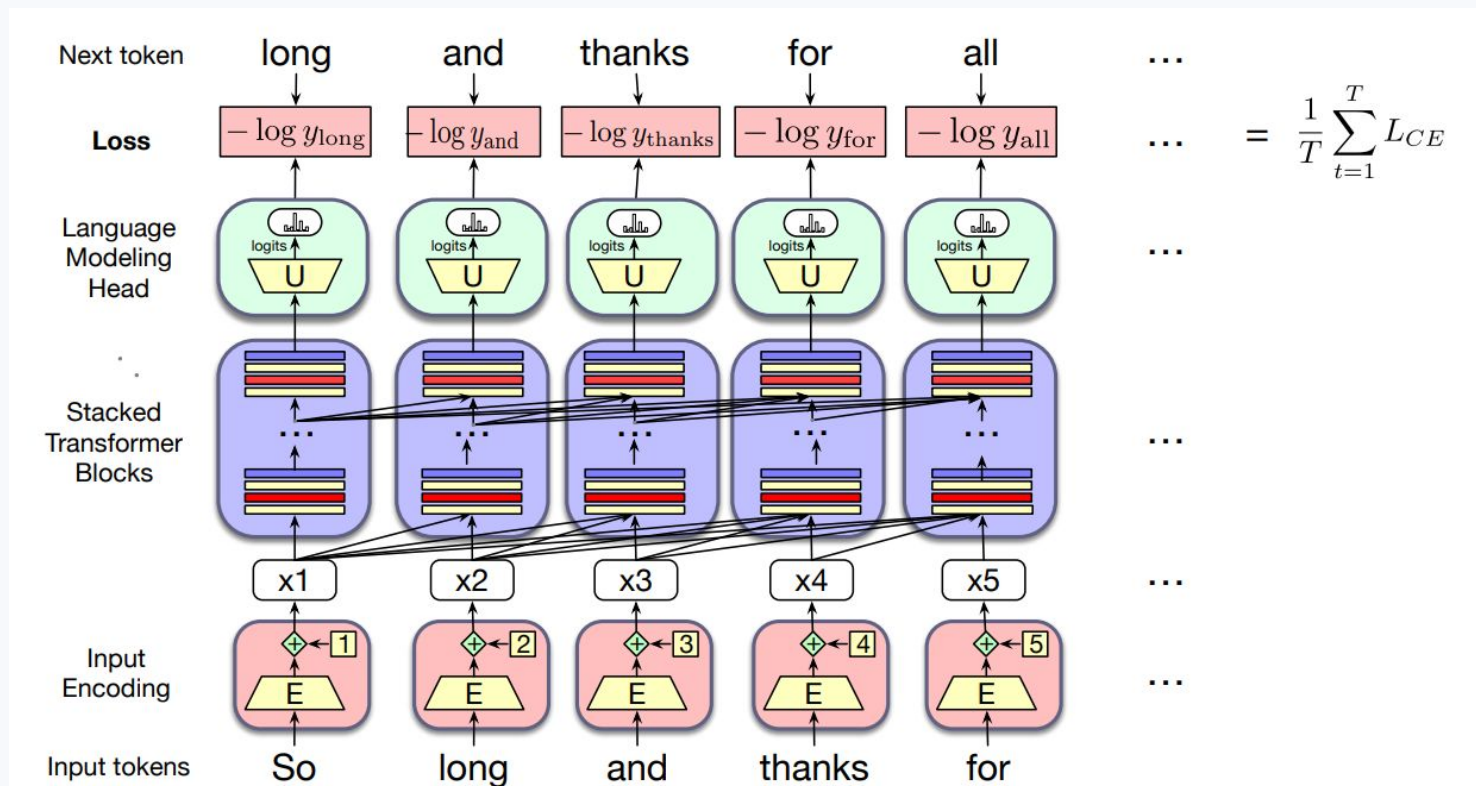
- At each token position t , model sees correct tokens $w_1:t$,
- Computes loss ($-\log$ probability) for the next token w_{t+1}
- At next token position $t+1$ we ignore what model predicted for w_{t+1}
- Instead we take the correct word w_{t+1} , add it to context, move on

Next Token Prediction

We just train them to predict the next token

1. Take a corpus of text
2. At each time step t
 - i. ask the model to predict the next token
 - ii. train the model using gradient descent to minimize the error in this prediction.

Training a transformer language model



👉 Fun fact: training GPT-5.5 likely involved training **trillions of params** with **~50,000 GPUs** over **~2–3 months** and cost around **\$100M–\$300M**.

Quiz

Scaling Laws

$$L(N) = \left(\frac{N_c}{N} \right)^{\alpha_N}$$

$$L(D) = \left(\frac{D_c}{D} \right)^{\alpha_D}$$

$$L(C) = \left(\frac{C_c}{C} \right)^{\alpha_C}$$

Loss L as a function of:

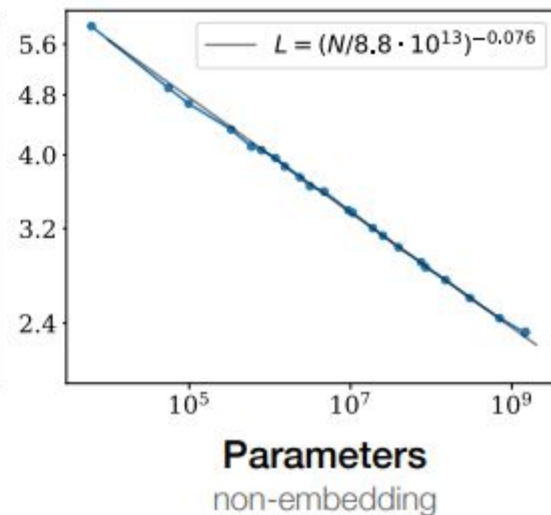
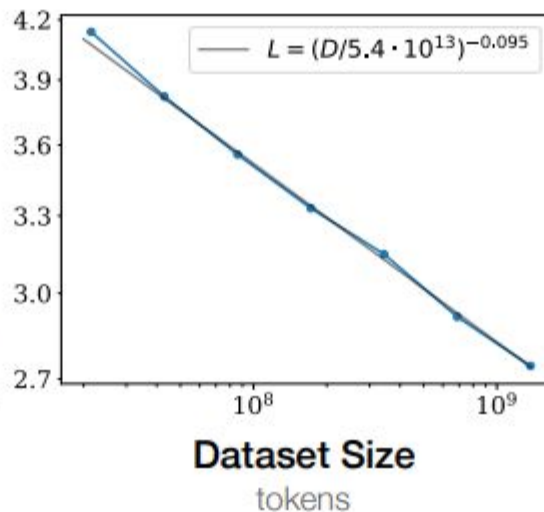
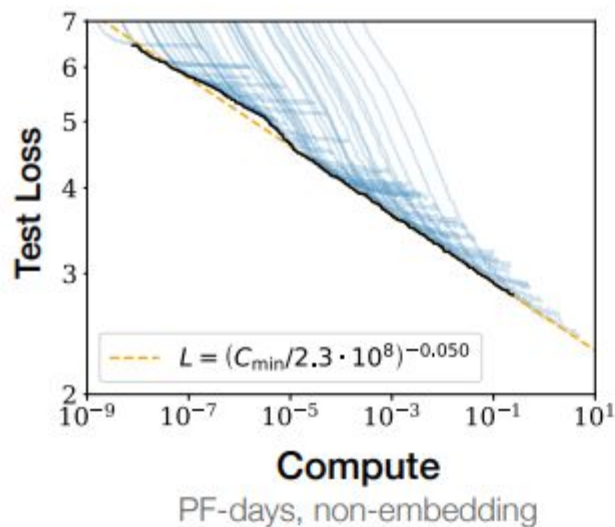
parameters N ,

dataset size D ,

compute budget C

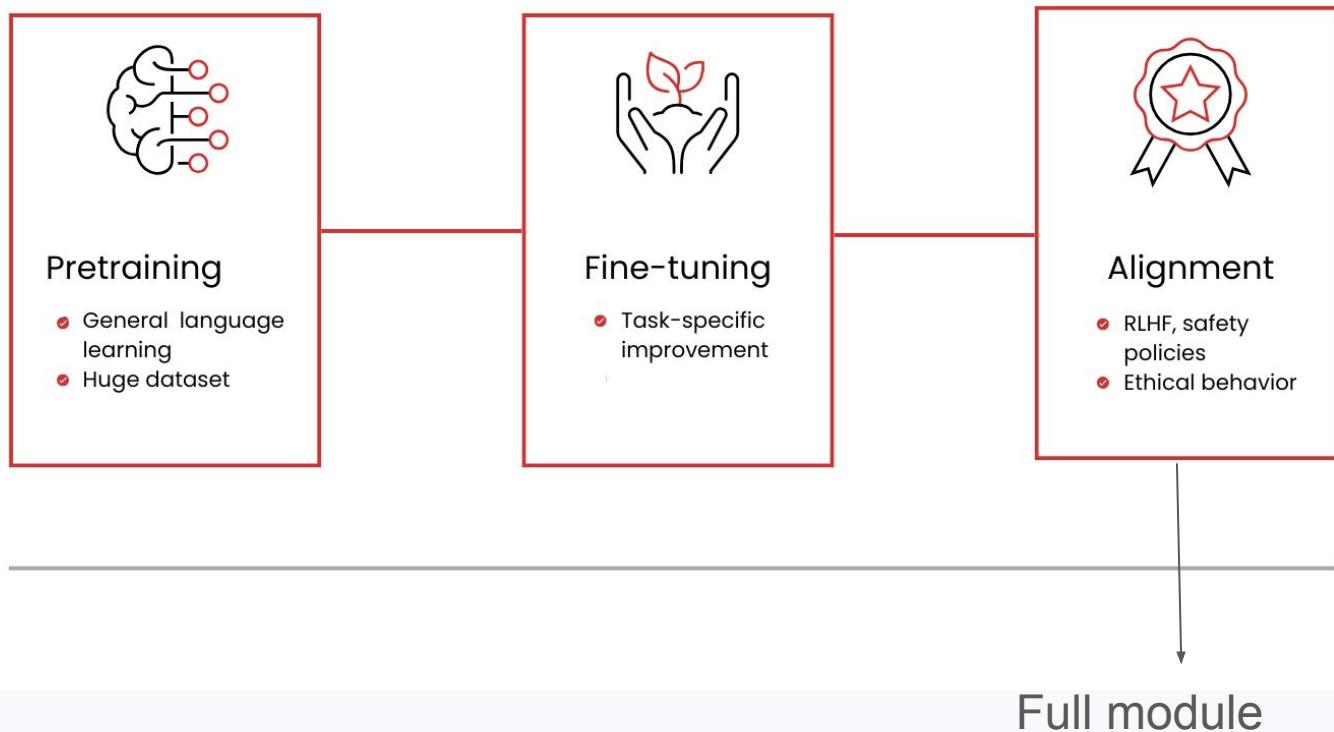
(if other if two are held constant)

Scaling Laws



Fine Tuning

LLM TRAINING PIPELINE

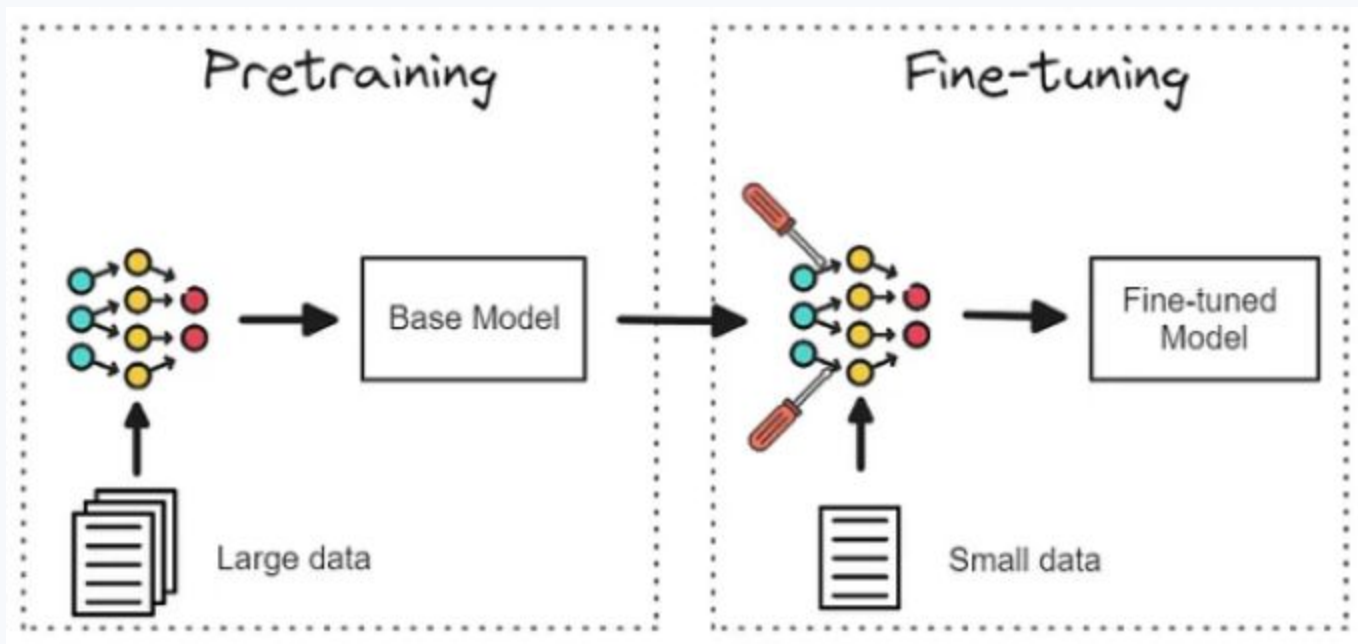


Pretrained Models Are Amazing But...

What happens if we need our LLM to work well on a domain it didn't see in pretraining?

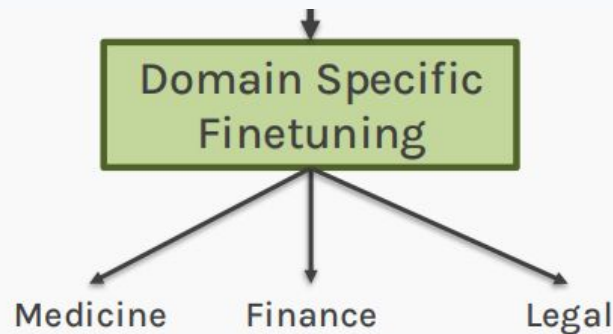
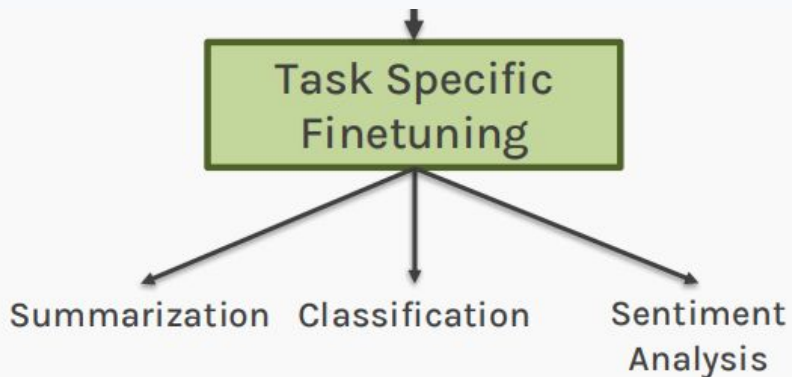
Perhaps some specific medical or legal domain?

Or maybe a multilingual LM needs to see more data on some language that was rare in pretraining?



Fine Tuning Definition:

taking a pretrained model and further adapting some or all of its parameters to some new data



When Should We consider Fine Tuning?

⚠ RAG solves a large portion of practical LLM use cases by grounding models in external knowledge.

Fine-tuning becomes valuable when you need to shape behavior, improve consistency, or optimize performance beyond what retrieval alone can provide.

Difference Between Fine Tuning and in Context Learning

Fine-tuning is a supervised process that leads to **a new model**, in contrast with in-context learning / few shot learning.

In-context Learning

- A method of prompt engineering where the model is shown task demonstrations as part of the prompt.
- No change in model

parameters.

Fine-tuning

- A process of training the LLM on a labelled dataset specific to a particular task.
- Change in model parameters.

Types of Fine Tuning

- Full Parameter Tuning
- PEFT
 - LoRA
 - QLoRA

Full Parameter Tuning

requires an instruction dataset, comprising pairs of instructions, answers, and sometimes context, is required for such fine-tuning.

Example:

Instruction	Context	Output
Suggest a good restaurant	Los Angeles, CA	In Los Angeles, CA, I suggest Rossoblu Italian Restaurant
Rewrite the sentence with more descriptive words	The game is fun	The game is exhilarating and enjoyable
Calculate the area of the triangle	Base: 5cm; Height: 6cm	The area of the triangle is 15 cm^2

Full Parameter Tuning

Task-specific fine-tuning: This particular process involves training the model on a smaller, task-specific dataset. For e.g.: Summarize this, translate that, etc

This allows the model to learn the nuances, and specialized vocabulary relevant to the task.

We have to be careful while doing task-specific finetuning to avoid **catastrophic forgetting**

To mitigate the problem of catastrophic forgetting, we need to do **multi-task finetuning**. This requires a lot of data, and training resources.

Full Parameter Tuning is Expensive!

- We need to update all the parameters while fine-tuning.

Adapting to a new domain by continued pretraining (finetuning) is a problem with huge LLMs.

- Enormous numbers of parameters to train
- Each pass of batch gradient descent has to backpropagate through many many huge layers.
- Expensive in processing power, in memory, and in time. .

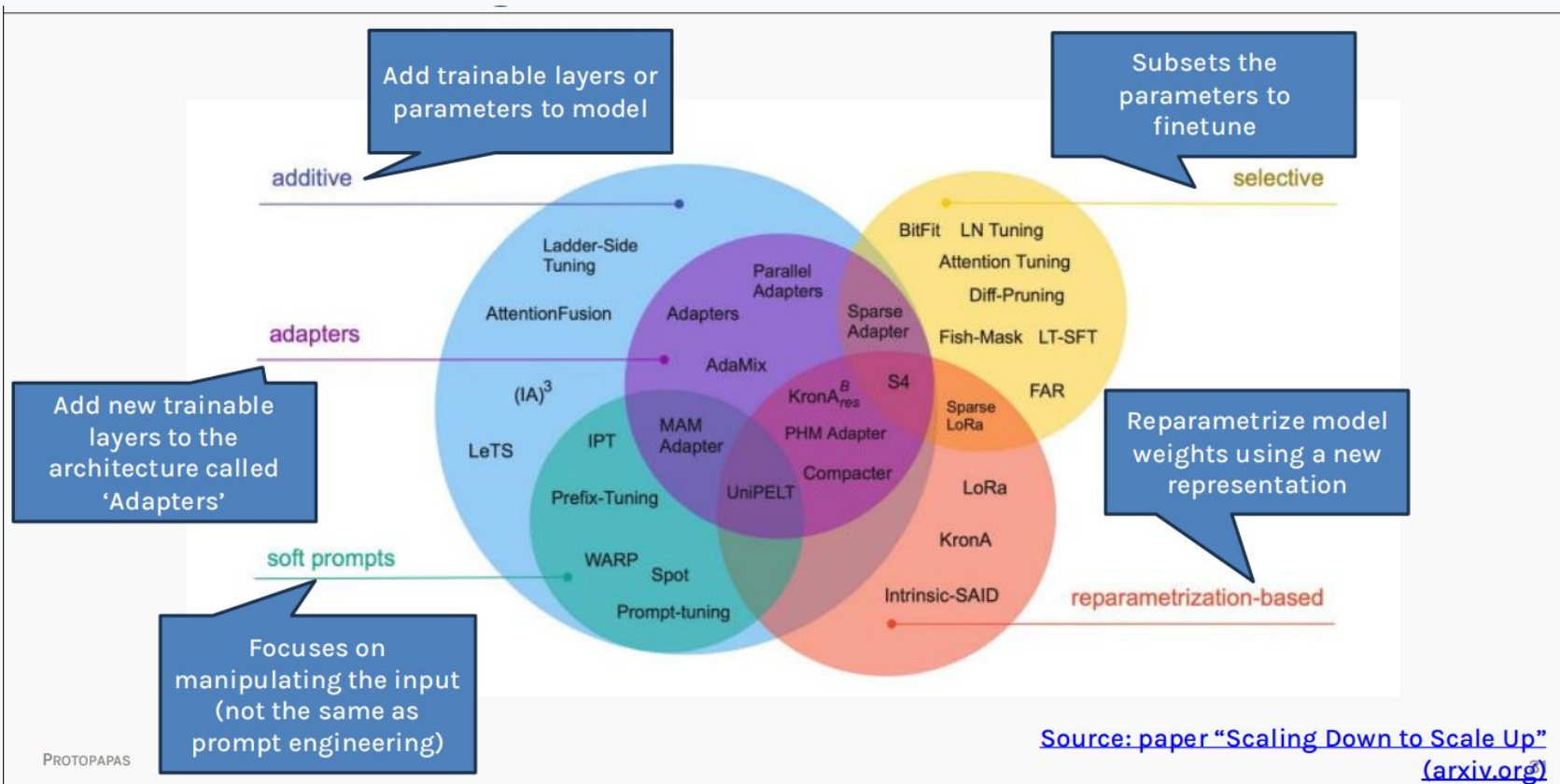
So what can we do?



Parameter-Efficient Finetuning PEFT

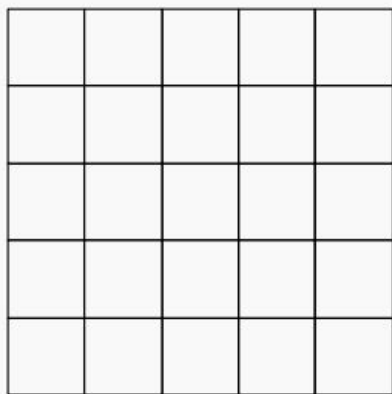
- Efficiently select a **subset of parameters** to update when finetuning.
- E.g., freeze some of the parameters (don't change them),
- And only update some a few parameters.

PEFT Types



LoRA Intuition

LoRA revolves around the idea that any matrix $W \in \mathbb{R}^{m \times n}$ can be decomposed into $W = BA$ where $B \in \mathbb{R}^{m \times r}$ and $A \in \mathbb{R}^{r \times n}$



W

=



B

×



A



Quiz

if w has 100 elements how many will A and B have with rank 1?



LoRA General Idea

- Transformers have many dense matrix multiply layers
 - Like WQ , WK , WV , layers in attention
- Instead of updating these layers during finetuning:
 - Freeze these layers
 - Update a low-rank approximation with fewer parameters.

Decomposing the Update Matrix ΔW

finetuning an LLM.

The diagram illustrates the decomposition of the update matrix ΔW in the context of finetuning an LLM. The central equation is $W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA$. Callouts identify the components: W_0 is the Initial LLM Weights, ΔW is the Update matrix, α is the Scaling parameter, r is the Rank of BA , and BA represents the Decomposed matrices.

$$W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA$$

LoRA

- Consider a matrix W (shape $[N \times d]$) that needs to be updated during finetuning via gradient descent.
- Normally updates are ΔW (shape $[N \times d]$)
- In LoRA, we freeze W and update instead a low-rank decomposition of W :
 - A of shape $[N \times r]$,
 - B of shape $[r \times d]$, r is very small (like 1 or 2)
 - That is, during finetuning we update A and B instead of W .
 - Replace $W + \Delta W$ with $W + BA$.

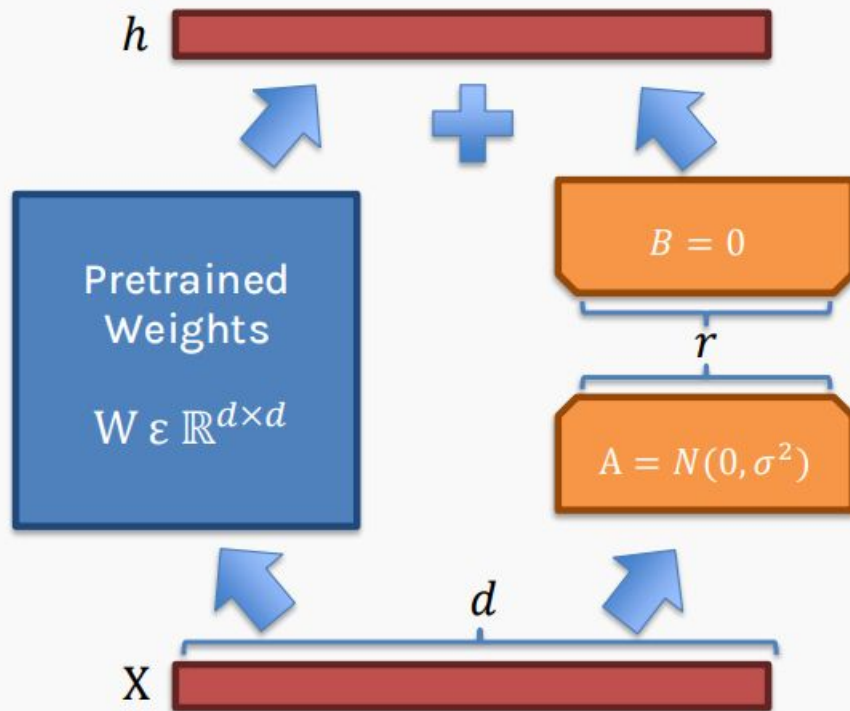
LoRA

Forward pass: instead of

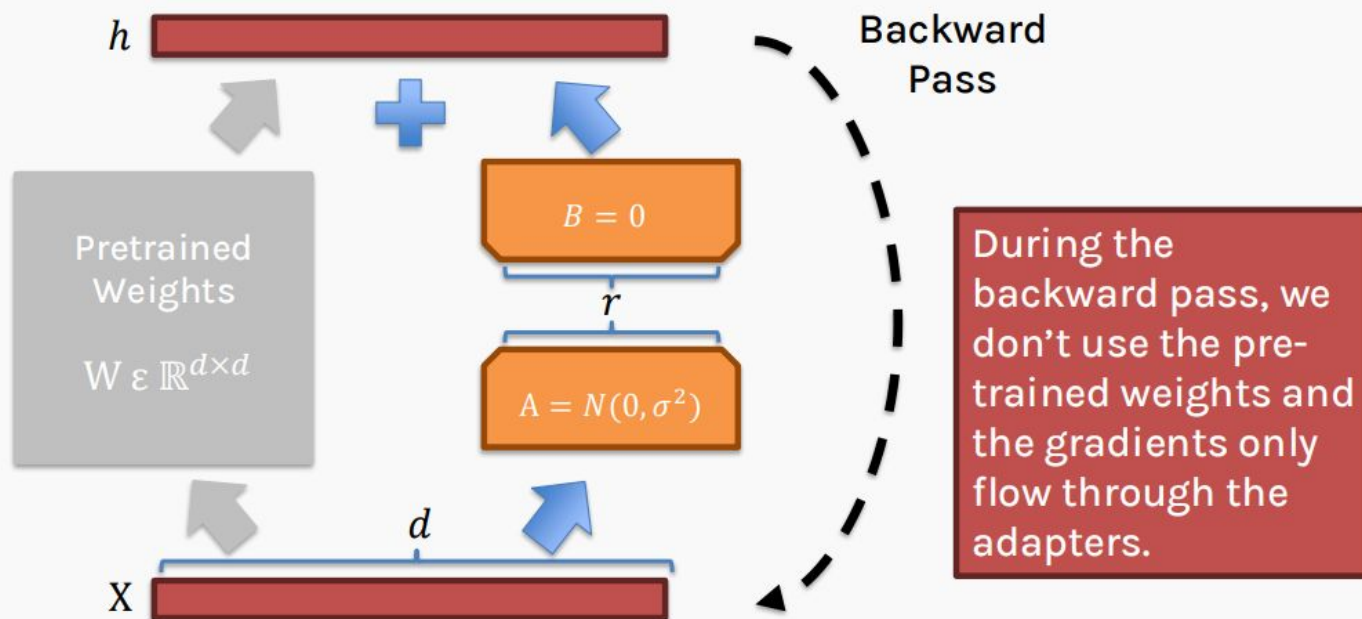
$$\mathbf{h} = \mathbf{x}\mathbf{W}$$

We do

$$\mathbf{h} = \mathbf{x}\mathbf{W} + \mathbf{x}\mathbf{A}\mathbf{B}$$



LoRA



LoRA's Impact

Rank	Model 7B	Model 13B	Model 70B	Model 180B
1	167K	228K	529K	849K
2	334K	456K	1M	2M
8	1M	2M	4M	7M
16	3M	4M	8M	14M
512	86M	117M	270M	434M
1024	171M	233M	542M	869M
8192	1.4B	1.8B	4.3B	7B
Full	7B	13B	70B	180B

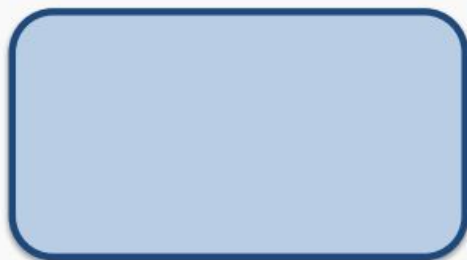
This is a generalization considering an LLM of one layer. LLMs are made up of multiple layers.

LoRA's Advantages

1. Much faster
2. Finetuning can be achieved using less GPU memory
3. Cost efficient
4. Less prone to “catastrophic forgetting” since the original model weights are kept the same

Full Parameter Fine Tuning

Optimizer
State
(FP32)



Base Model
(FP16)



10B → 160GB

LoRA

Optimizer
State
(FP32)



LoRA
Adapter
(FP16)



Base Model
(FP16)



10B → ~40GB

LoRA even Often Outperforms Full Fine Tuning

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4 $\pm$.1	8.85 $\pm$.02	46.8 $\pm$.2	71.8 $\pm$.1	2.53 $\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49 $\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4 $\pm$.1	8.89 $\pm$.02	46.8 $\pm$.2	72.0 $\pm$.2	2.47 $\pm$.02

Table 3: GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments we ran. * indicates numbers published in prior works.

Can We Do Better? QLoRA

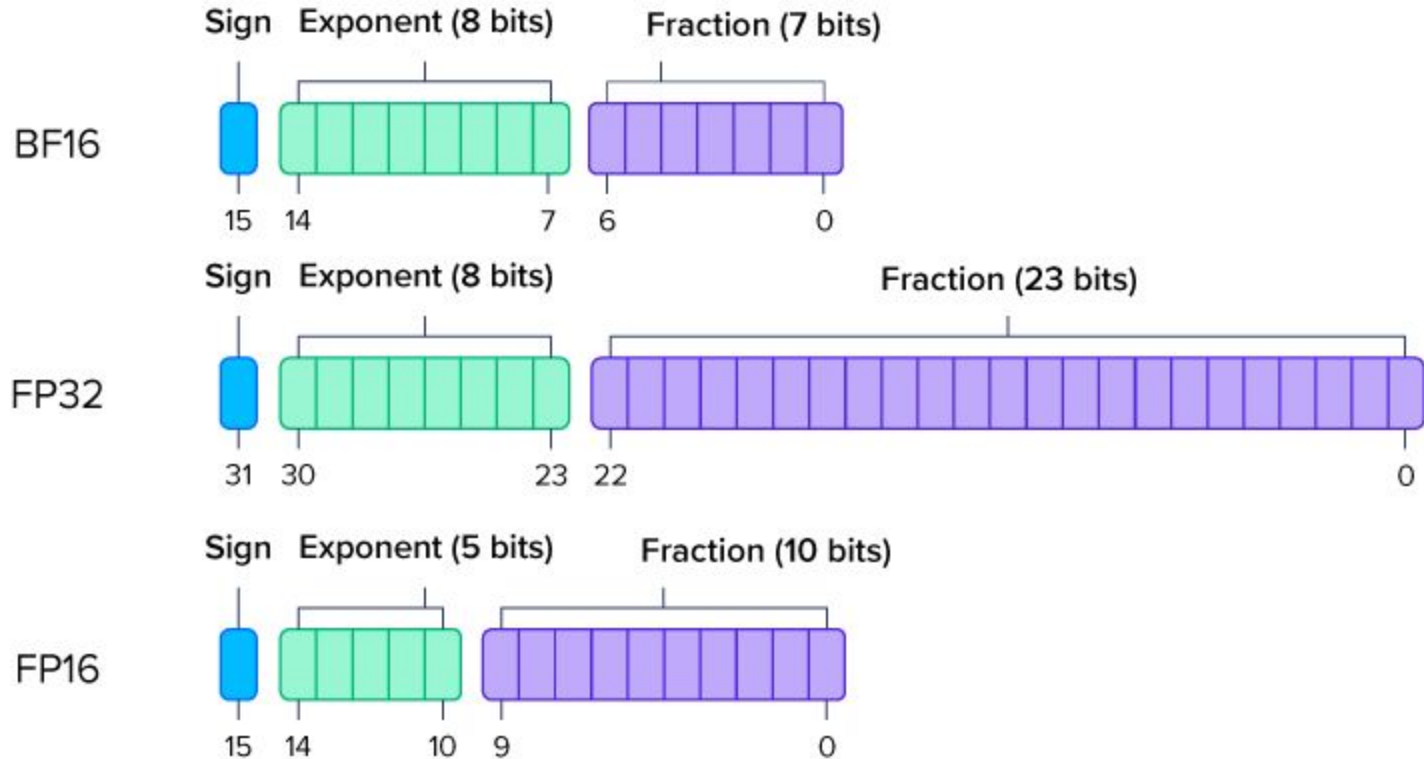
QLoRA is the extended version of LoRA which works mainly by quantizing the precision of the network parameters.

Types

FIGURE 2.1 Primitive Type

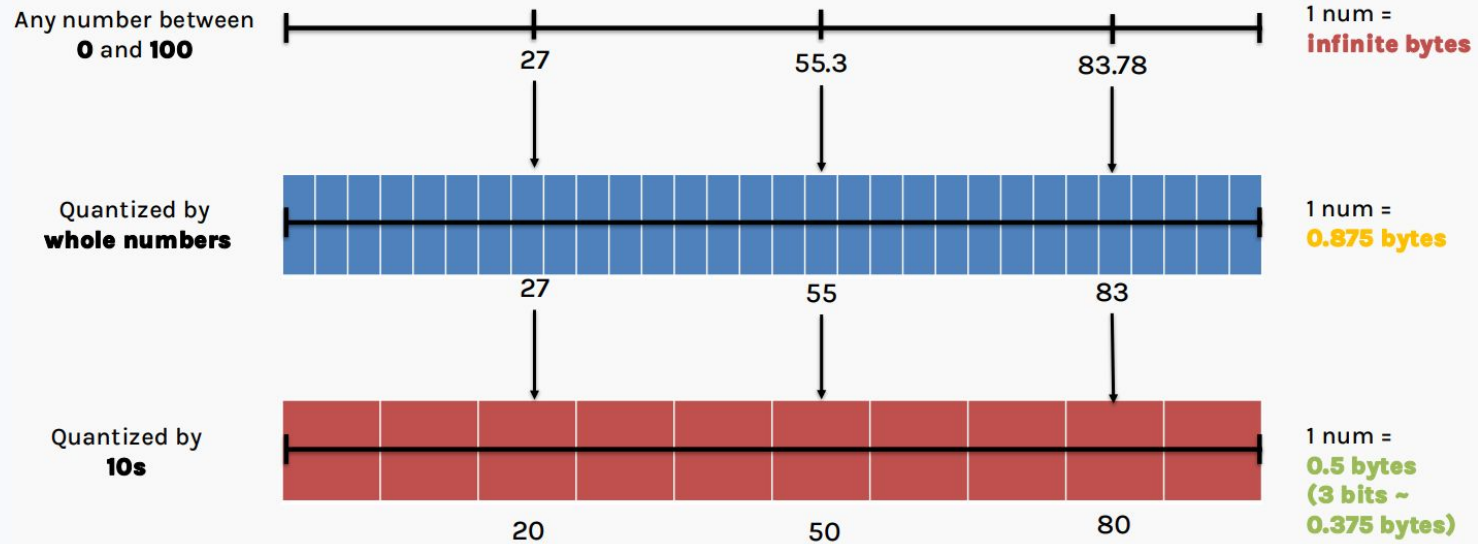
Type Name	Kind of Value	Memory Used	Range of Values
byte	Integer	1 byte	−128 to 127
short	Integer	2 bytes	−32,768 to 32,767
int	Integer	4 bytes	−2,147,483,648 to 2,147,483,647
long	Integer	8 bytes	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	Floating-point	4 bytes	$\pm 3.40282347 \times 10^{+38}$ to $\pm 1.40239846 \times 10^{-45}$
double	Floating-point	8 bytes	$\pm 1.79769313486231570 \times 10^{+308}$ to $\pm 4.94065645841246544 \times 10^{-324}$
char	Single character (Unicode)	2 bytes	All Unicode values from 0 to 65,535
boolean		1 bit	True or false

Float Types



Quantization

Think of quantization as 'splitting range into buckets'.



QLoRA Example

What if we want to quantize from FP32 to Int8?

1.5

2.3

3.7

4.1

5.6

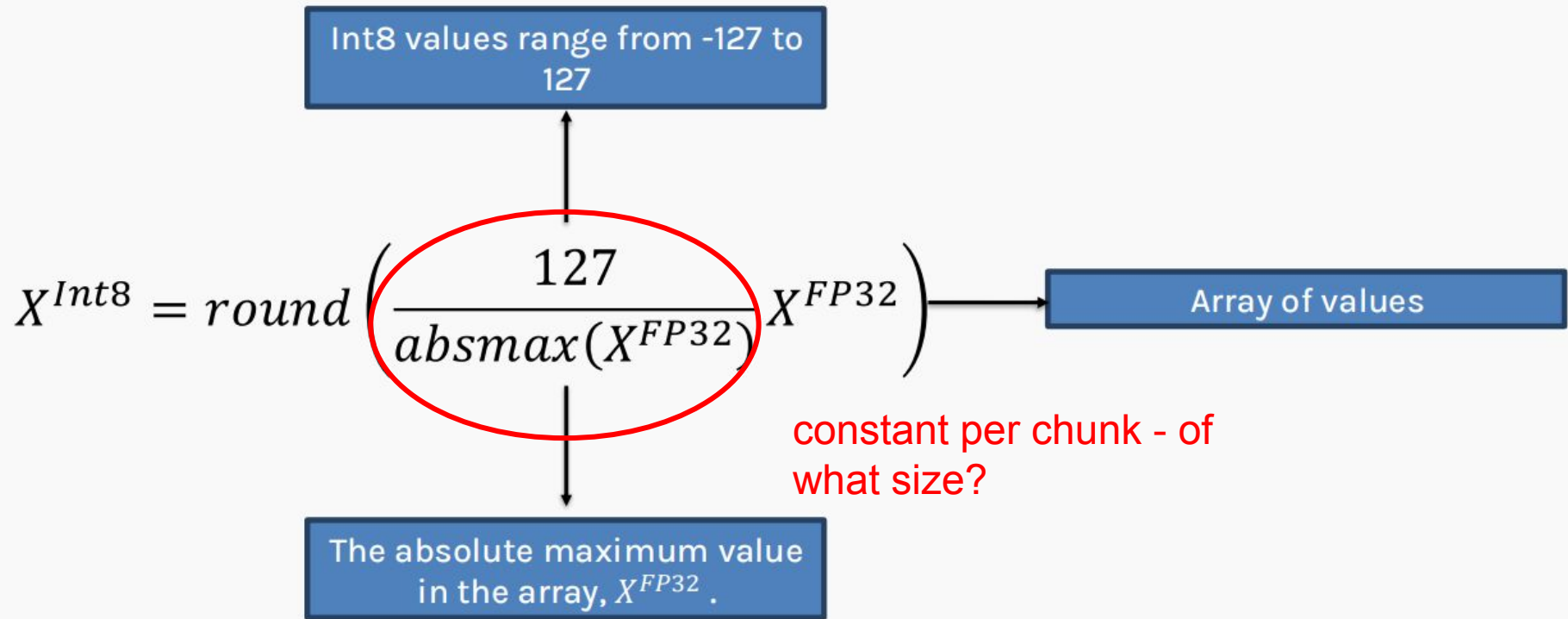
6.8

7.9

8.4

9.2

10.2



1.5

2.3

3.7

4.1

5.6

6.8

7.9

8.4

9.2

10.2



$$X^{Int8} = round\left(\frac{127}{absmax(X^{FP32})} X^{FP32}\right)$$



$$X^{Int8} = round(c^{FP32} X^{FP32})$$

18

29

46

51

69

85

98

105

115

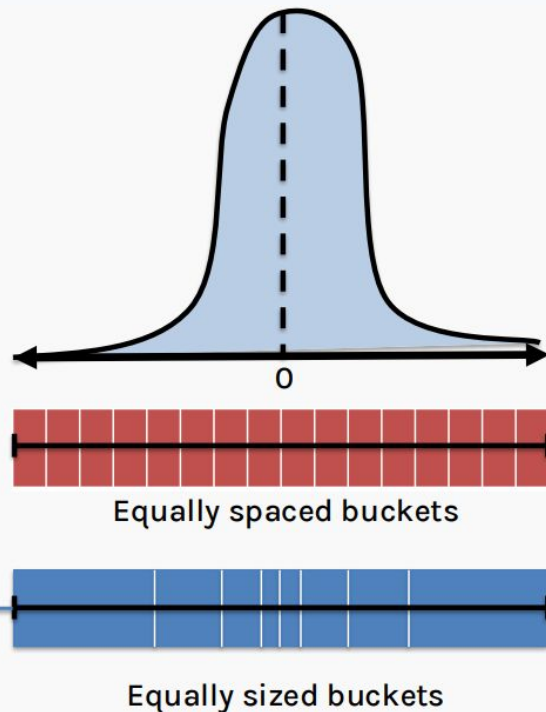
127

QLoRA - We normally do 4-bit

4-bit NormalFloat

4-bit NormalFloat is a clever way to split the buckets.

4-bit means we have $2^4 = 16$ possible buckets for quantization.



This is an enhanced version of **quantile quantization**.

What if we want to dequantize and get back the original array, X^{FP32} ?

$$X^{FP32} = \frac{X^{Int8}}{c^{FP32}}$$

with some dequantization error

QLoRA

What can be the problem with quantization of large matrices?

The constant can be defined by an outlier.

What do we do?

We flatten the matrix. Now we divide it up into different blocks.

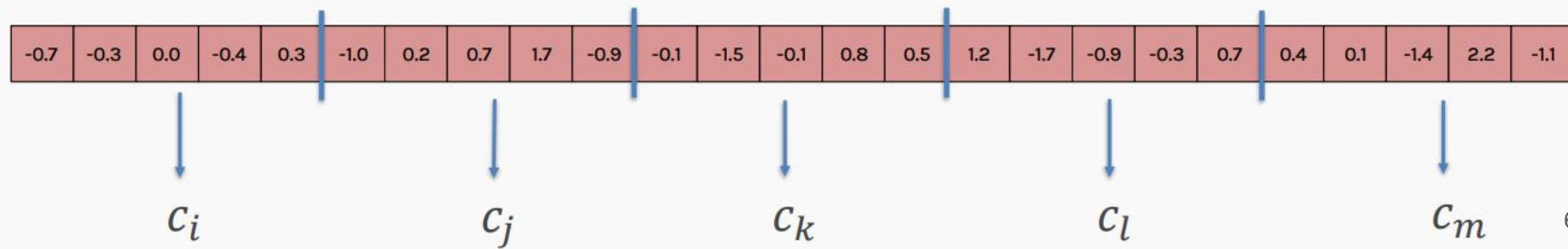
We calculate the quantization constants for each block.

Weight Tensor (W^{FP32})

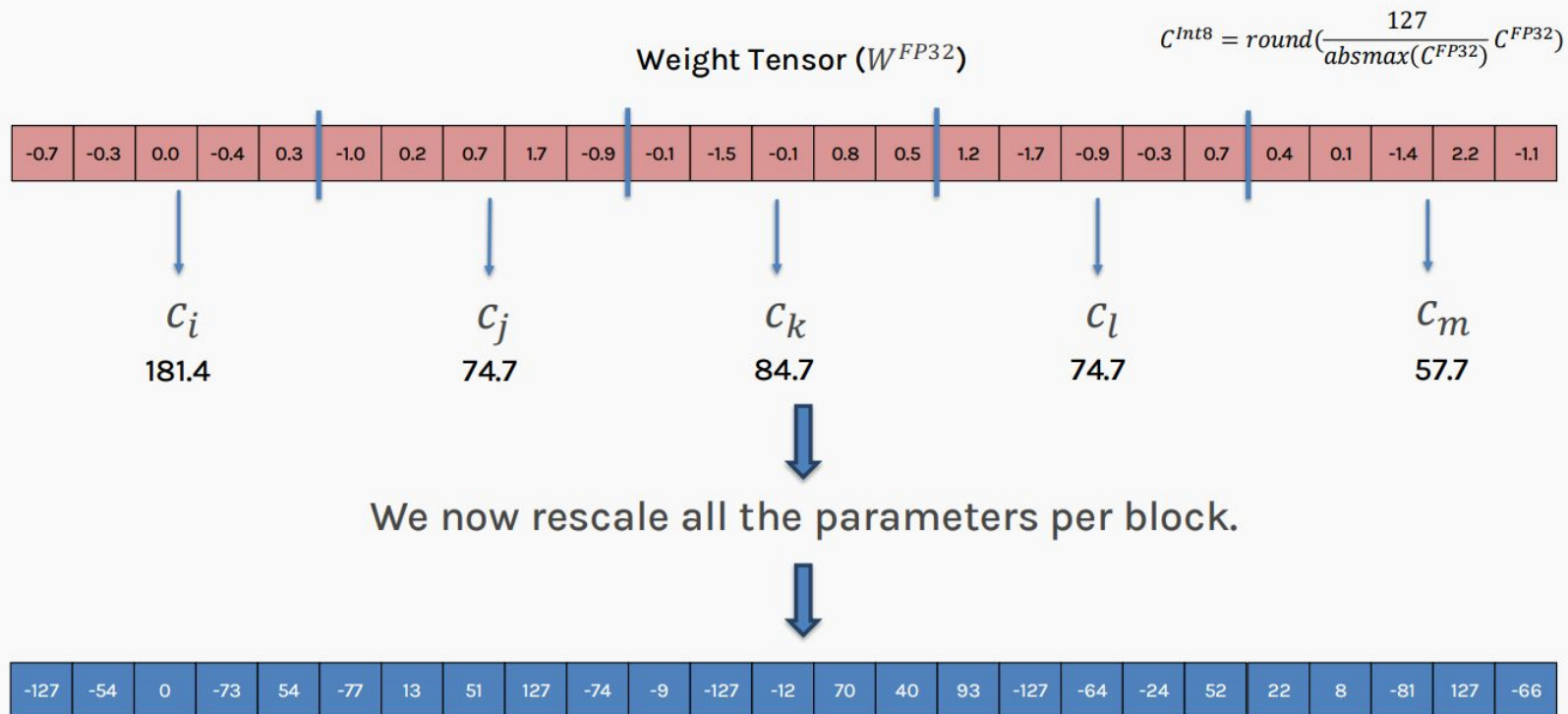
-0.7	-0.3	0.0	-0.4	0.3
-1.0	0.2	0.7	1.7	-0.9
-0.1	-1.5	-0.1	0.8	0.5
1.2	-1.7	-0.9	-0.3	0.7
0.4	0.1	-1.4	2.2	-1.1



Weight Tensor (W^{FP32})

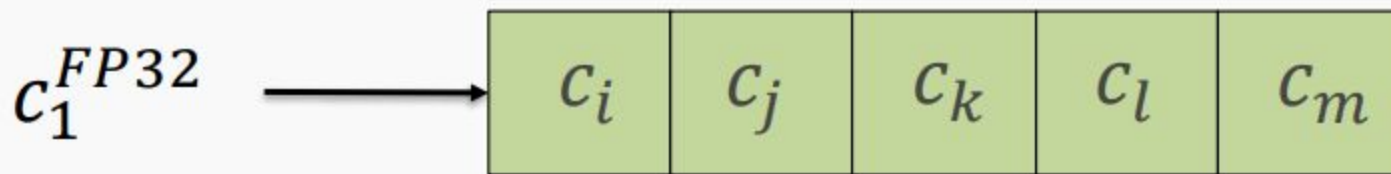


QLoRA: Double Quantization



QLoRA: Double Quantization

Now we have a new array



We'll also quantize this

$$\left. \begin{aligned} c_1^{Int8} &= \text{round}\left(\frac{127}{\text{absmax}(c_1^{FP32})} c_1^{FP32}\right) \\ c_1^{Int8} &= \text{round}(c_2^{FP32} c_1^{FP32}) \end{aligned} \right\} \text{Double Quantization}$$

Quiz

Before:

All we had was a weight matrix containing FP32 values. In our example, we had a 5x5 matrix. Each value was 4 bytes in size. So, the total memory used $25 \times 4 = 100$ bytes

What will have after with double quantization to int8 = 1 bytes?



-0.7	-0.3	0.0	-0.4	0.3
-1.0	0.2	0.7	1.7	-0.9
-0.1	-1.5	-0.1	0.8	0.5
1.2	-1.7	-0.9	-0.3	0.7
0.4	0.1	-1.4	2.2	-1.1

Weight Tensor (W^{FP32})

Answer

-127	-54	0	-73	54
-77	13	51	127	-74
-9	-127	-12	70	40
93	-127	-64	-24	52
22	8	-81	127	-66

Rescaled Weight Tensor
(W^{Int8})

25x1=25 bytes.

c_i	c_j	c_k	c_l	c_m
-------	-------	-------	-------	-------

c_1^{Int8}

5x1=5bytes.

c_2^{FP32}

4 bytes

So, in total:

25 + 5 + 4 = **34 bytes**

Summary

1. There are three steps in training LLMs: Pre-training, Fine tuning and Alignment
2. Common methods to fine tune a model is by using LoRA or QLoRA